

ReconIQ — Project Summary

Repo: <https://github.com/Shaurya55555/reconiq> **Live demo:** <https://reconiq-mocha.vercel.app> **Architecture diagram:** <docs/architecture.html>

Built for the Razorpay AI Buildathon — **Track 04: AI Finance Controller.**

What Track 4 asked for

"Build an agent that closes one finance-ops loop across a 50+ record batch of synthetic data, reporting its match rate and the exceptions it could not resolve."

The bar (their exact words):

"Throughput plus measured accuracy plus an honest exception list. One cherry-picked match proves nothing."

What ReconIQ is

A multi-source reconciliation agent: it matches an internal order ledger against a Razorpay settlement export and a raw bank statement — the way a finance-ops person would by hand, except verified, auditable, and able to handle the messy cases most reconciliation tools give up on. It doesn't try to match everything — it matches what it can prove, escalates what deterministic rules can't resolve to an LLM behind a confidence gate, and leaves genuine uncertainty as an honest, reason-coded exception.

The matching engine — passes, cheapest and most certain first

1. **Exact match** — settlement UTR found verbatim in a bank line, amount matches to the paisa.
2. **Fuzzy match** — UTR found, amount differs by a plausible Razorpay fee ($\leq 3\%$, configurable) or the settlement date drifted.
3. **Group matching, both directions (1:N and N:1)** — one settlement's money arriving as 2–3 separate bank credits (`group_split`), or several orders' payments consolidated into one settlement and one bank credit (`batch_settlement`, genuinely how Razorpay's batch settlement works). Both fully deterministic — the shared UTR proves the group belongs together, not an LLM guess.
4. **Refund-aware matching, timing-sensitive** — a full refund resolves with no settlement expected (`method: "refunded"`); a partial refund is matched against `order.amount - refund.amount`, not the raw order amount. Critically, netting only applies to a refund that predates its settlement (`refund.refunded_at <= settlement.settled_at`) — a refund issued *after* settlement is a separate, later cash event and must never make an already-legitimate full settlement look like an `amount_mismatch`. Computed per settlement candidate, not once per order, so this is correct even with multiple candidate settlements.
5. **Cash-position matching** — a refund record proves money was promised back, not that it left the account. Every refund is also matched against its own outbound (negative-amount) bank debit line; one with no matching debit becomes an honest `refund_not_debited` exception carrying its own amount, counted toward amount at

risk and the closing verdict instead of being assumed to have gone out.

6. **LLM-resolved match** — only when the UTR isn't found in *any* bank line at all (garbled/truncated narration). A real Gemini call reasons over the free text and returns a confidence score; an offline heuristic fallback covers both the no-API-key case and any live-call failure (rate limit, timeout) — gracefully, never a crash.
7. **Confidence gate** — an LLM verdict only auto-clears if its confidence is at or above a tunable threshold. Below it, the case becomes an honest exception instead of a guess.
8. **Honest exceptions** — anything still unresolved gets a specific reason code, never dropped silently.

Everything built on top of the core loop

- **Ground-truth accuracy** — scored against a hidden truth label the matcher never sees, distinct from raw match rate.
- **Value-weighted accuracy** — order-level correctness weighted by rupee amount, not just transaction count.
- **False-clear vs. safe-miss split** — every wrong answer is categorized as dangerous (confidently wrong) or conservative (flagged for review), with a measured rupee amount for each.
- **Rules-only vs. Rules+AI comparison** — computed on the *same batch* every run, proving what the AI layer actually contributes instead of asserting it.
- **Closing verdict** — a plain "Safe to close" / "Review before closing" call synthesized from materiality-weighted exceptions (■X remains unresolved, including ■Y above your materiality threshold), so a finance user doesn't have to read every exception row to know whether the books can close today.
- **Exception priority** — every exception is tagged high/medium/low by rupee amount relative to a tunable materiality threshold, sorted so the exceptions that actually matter are reviewed first.
- **Plain-English policy presets** — Conservative / Balanced (Recommended) / High Automation, labeled by business tradeoff, not by "more AI"; exact numeric values stay visible in an Advanced panel for a technical reviewer.
- **Empirical confidence-threshold calibration** — `POST /api/calibrate` sweeps the LLM auto-accept threshold across a range of values on one fixed batch and one fixed round of LLM verdicts, reporting match rate, accuracy, and false-clear ■ at each point, so the default (0.6) can be defended with a number instead of asserted.
- **Multi-seed corruption benchmark** — `POST /api/benchmark` runs several independent batches per corruption level and reports the mean with its min-max spread, so "accuracy holds up as data gets messier" is backed by several runs per point, not one cherry-picked batch.
- **Human-in-the-loop override** — accept / reject / manually-match any decision, logged distinctly as `human_override` in the audit trail so it's never confused with an automated call.
- **Per-decision evidence drawer** — click "Why?" on any match or exception to see the full lineage (order/settlement/bank amounts, UTR vs. narration, date drift, refund detail, AI involvement, actual reasoning).
- **Bring-your-own-data** — upload real CSVs (orders, settlements, bank lines, and an optional refunds file) in the same schema and run the identical pipeline against them, not just synthetic data. No fabricated ground-truth accuracy is shown for uploaded data.
- **Full audit trail**, exportable as CSV, for every decision at every stage.
- **Hero-first dashboard** — leads with financial position (amount processed / reconciled / at risk / refunded), the closing verdict banner, and a one-line interpretation note, with technical policy knobs de-emphasized into an "Advanced settings" panel rather than hidden.

Canonical demo batch (seed 42, 100 orders)

Locked for the pitch video and repeatable by anyone: `POST /api/run {"n_orders": 100, "seed": 42}`

Metric	Value
Match rate	92% (92/100)
Ground-truth accuracy	98%
Value-weighted accuracy	97.7%
False-clear amount	■0
Safe-miss amount	■29,916
Amount processed	■12,09,087
Amount reconciled	■9,68,871
Amount at risk	■2,40,216
Amount refunded	■1,81,916
Rules-only match rate	90%
Rules+AI match rate	92% (+2.0pp AI uplift, same batch)
Rules-only accuracy	96% → Rules+AI accuracy: 98%
Methods represented	exact, fuzzy, group_split, batch_settlement, refunded, llm — all six, plus a refunded_after_settlement case (settlement matches at full value, refund tracked separately)
Closing verdict	Review before closing — ■2,40,216 unresolved, ■2,24,651 above materiality threshold (■5,000)

This single batch exercises every matching pass, the refund-aware matcher (including the pre- vs. post-settlement refund distinction), the AI layer, and the closing verdict — chosen deliberately for that diversity, not cherry-picked for a flattering number. The 2% gap from perfect accuracy here is an honest **safe miss** (a case flagged for review that was actually fine — the conservative failure mode, never a false clear) — not tuned away, because the multi-seed benchmark below is what actually backs the accuracy claim, not this one batch.

Benchmark — multi-seed, run live against production

`POST /api/benchmark` runs `n_seeds` independent batches per corruption level (not one) and reports the mean with its min–max spread, so the accuracy claim is backed by several runs per point, not a single cherry-picked batch.

Pulled live from production (`{"n_orders": 70, "corruption_rates": [0.1,0.2,0.3,0.4,0.5], "n_seeds": 3, "use_llm": true}`), real Gemini calls, 15 independent batches total):

Corruption	Match rate (range)	Verified acc. (range)	Value-weighted acc.	False-clear ■
10%	97.6% (95.7–100.0%)	99.5% (98.6–100.0%)	99.6%	■0
20%	96.7% (95.7–97.1%)	100.0%	100.0%	■0
30%	95.2% (94.3–97.1%)	99.5% (98.6–100.0%)	99.1%	■0
40%	91.9% (90.0–92.9%)	98.6% (97.1–100.0%)	99.1%	■0
50%	90.0% (87.1–92.9%)	100.0%	100.0%	■0

False-clear amount was ■0 — min and max, across all 3 seeds — at every corruption level tested, including 50%. The system never once confidently matched something it should have flagged, across a genuinely harsh, multi-seed stress test. Match rate declines at higher corruption because more corruption correctly produces more honest exceptions — that's the system behaving correctly, not degrading.

Confidence-threshold calibration — proven, not asserted

Pulled live from production (`{"n_orders": 150, "seed": 917}`), 6 LLM-deferred cases in this batch, real Gemini):

Threshold	Match rate	Verified acc.	LLM cases accepted	False-clear ■	Safe-miss ■
0.5	90.0%	100.0%	6/6	■0	■0
0.6 (default)	90.0%	100.0%	6/6	■0	■0
0.7	90.0%	100.0%	6/6	■0	■0
0.8	89.3%	99.3%	5/6	■0	■5,086
0.9	88.0%	98.0%	3/6	■0	■32,522

The default (0.6) sits inside the range that keeps false-clear at ■0 while maximizing coverage — tightening past 0.7 only trades away coverage (more safe-miss ■) for no additional safety, since false-clear never appears in this sweep at any threshold tested. That's the empirical basis for the default, not an assertion.

Proof it works

- **77/77 automated tests passing**, including dedicated adversarial tests: group-split/batch-settlement matching never falsely combines unrelated bank lines or rescues a genuine amount mismatch; a refund only ever explains the gap it actually accounts for and never masks an unrelated discrepancy; a refund issued *after* settlement never shrinks an already-legitimate settlement, while one issued *before* still nets correctly (both directions locked in, not just the happy path); a stricter confidence threshold never auto-accepts *more* LLM matches than a looser one.
 - **CI green** on every commit (GitHub Actions).
 - **Verified live in a browser**: real clicks (not just API calls) — running a reconciliation, opening the evidence drawer, performing a human override, running the calibration sweep and multi-seed benchmark, confirming the audit trail — all working end to end against the actual production URL.
 - **Real Gemini integration**, not a mock, with a tested offline heuristic fallback for both the no-API-key case and any live-call failure (rate limit, timeout, provider outage) — verified live: a Gemini 429 during testing fell back cleanly with the fallback reasoning explicitly tagged in the audit trail, never a silent guess or a crash.
 - **Release-candidate tag** (`reconiq-v1.0-rc`) marking the frozen, fully-verified engineering state ahead of pitch/demo prep.
-

Honest limitations, stated up front

1. **Scale ceiling on the live demo.** Very large batches (~200+ orders with many LLM calls) can occasionally hit Vercel's serverless duration cap or Gemini rate-limit delays under heavy usage. The deterministic matcher itself runs in under a millisecond regardless of batch size — any slowness is network/LLM latency, not the core engine.
 2. **True many-to-many isn't built.** Both group-matching directions are built (1:N `group_split`, N:1 `batch_settlement`); arbitrary many-to-many (several settlements partitioned against several bank lines simultaneously) is a meaningfully harder combinatorial problem and isn't attempted — noted explicitly, not quietly implied.
 3. **Chargebacks aren't built.** Refunds (full and partial) are; chargebacks are a different animal (bank/network-initiated, adversarial, their own dispute lifecycle) and are deliberately scoped out of v1.
 4. **Deliberately out of scope, on purpose:** cash forecasting, reconciliation aging as a standalone view, a general-purpose finance chatbot, payroll, invoicing, tax workflows, expense management, or any other accounting product. Track 4 asks for one loop done credibly, not a broader finance suite — every one of those would dilute that story rather than strengthen it.
 5. Ground-truth accuracy on synthetic data is high partly because the matcher's tolerance thresholds and the generator's anomaly ranges were authored by the same person, together — this demonstrates the pipeline is internally consistent, not that it's proven against arbitrary real-world noise. Bring-your-own-data mode exists precisely so real data can be run through the same pipeline, honestly without a fabricated accuracy number.
-

Offline Razorpay Settlement API adapter

`backend/scripts/fetch_razorpay_settlements.py` fetches real settlement data from Razorpay's Settlement Recon Details API and normalizes it into the same schema the bring-your-own-data upload already accepts — proof the engine consumes real Razorpay data, not just synthetic. Deliberately an offline script, not a live "Connect Razorpay" button in the deployed app: no external network dependency in the judging path, no live credentials in production. Isolated from the reconciliation engine by design (authenticate → fetch → paginate → normalize →

write CSV only, never imports `matcher.py`), with 4 dedicated unit tests covering single-payment settlements, multi-payment settlements (written out explicitly with a warning, never silently dropped), refund/transfer-row exclusion, and paise-to-rupee conversion. See the README for usage.

Stack

Python / FastAPI backend, stateless API (every response is self-contained — survives serverless cold starts by design), static HTML/JS dashboard (no build step), deployed to Vercel as a Python serverless function. Docker + docker-compose for local one-command run.

What's left

Engineering is frozen at `reconiq-v1.0-rc`. Remaining: architecture diagram (done — `docs/architecture.html`), pitch script (done — `docs/pitch_video_script.md`), judge Q&A prep (done — `docs/judge_qna.md`), record the 5-minute video, fill out the application form, submit.